

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 6

“Double Linked List”



OLEH:

LEXI MULIA YUNASPI

2511531006

**MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : DR WAHYUDI,MT**

ASISTEN PRAKTIKUM : M GHALID

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026**

KATA PENGANTAR

Puji syukur penulis ucapkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data mengenai *Doubly Linked List (DLL)* ini dapat diselesaikan dengan baik. Laporan ini dibuat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam praktikum ini, penulis mempelajari konsep dasar Doubly Linked List seperti proses pembuatan node, penambahan data, penghapusan data, penelusuran maju dan mundur, serta penerapannya pada program playlist musik menggunakan bahasa pemrograman Java.

Melalui praktikum ini penulis dapat memahami bagaimana struktur data DLL bekerja secara dinamis. Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan selanjutnya dapat menjadi lebih baik. Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi pembaca dan dapat menambah pemahaman mengenai struktur data Doubly Linked List.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Langkah Langkah Praktikum	2
2.2 Langkah Langkah Tugas Praktikum.....	44
BAB III PENUTUP	45
3.1 Kesimpulan.....	45
3.2 Saran.....	45
DAFTAR PUSTAKA	46

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu materi penting dalam pemrograman karena digunakan untuk mengatur dan mengelola data agar lebih terstruktur. Salah satu struktur data yang sering digunakan adalah *Linked List*. *Linked List* memiliki beberapa jenis, salah satunya yaitu *Doubly Linked List (DLL)*. *Doubly Linked List* adalah struktur data yang memiliki dua pointer, yaitu pointer menuju node berikutnya (*next*) dan pointer menuju node sebelumnya (*prev*). Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari depan maupun dari belakang. Struktur data ini lebih fleksibel dibandingkan *singly linked list* karena mempermudah proses penambahan dan penghapusan data. Pada praktikum ini dilakukan pembuatan beberapa program menggunakan Java, seperti pembuatan node DLL, penelusuran maju dan mundur, operasi insert, delete, serta implementasi playlist musik sederhana menggunakan konsep *Doubly Linked List*. Praktikum ini bertujuan agar mahasiswa memahami cara kerja DLL secara langsung melalui implementasi program.

1.2 Tujuan Praktikum

1. Memahami konsep dasar *Doubly Linked List*.
2. Mengetahui cara membuat node pada DLL menggunakan Java.
3. Memahami proses penambahan dan penghapusan node pada DLL.
4. Memahami proses traversal maju dan mundur pada DLL.
5. Mengimplementasikan DLL pada program playlist musik sederhana.

1.3 Manfaat Praktikum

1. Menambah pemahaman mahasiswa mengenai struktur data dinamis.
2. Melatih kemampuan mahasiswa dalam membuat program Java berbasis *linked list*.
3. Membantu mahasiswa memahami proses insert, delete, dan traversal data.

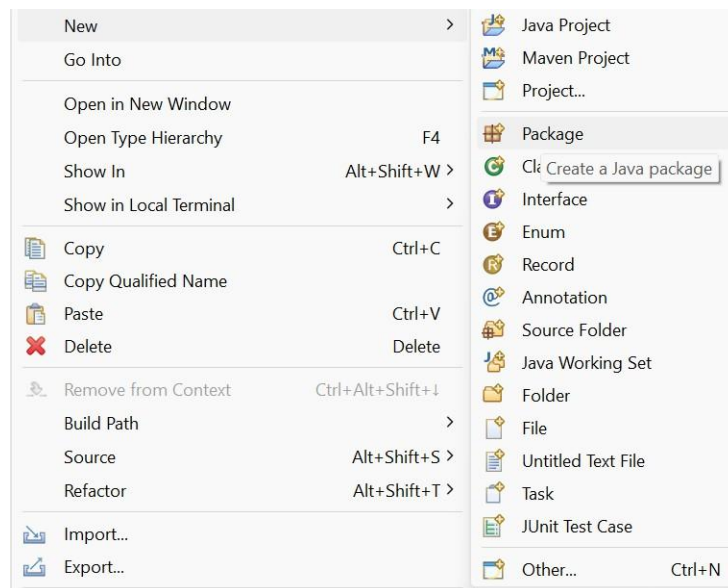
BAB II

PEMBAHASAN

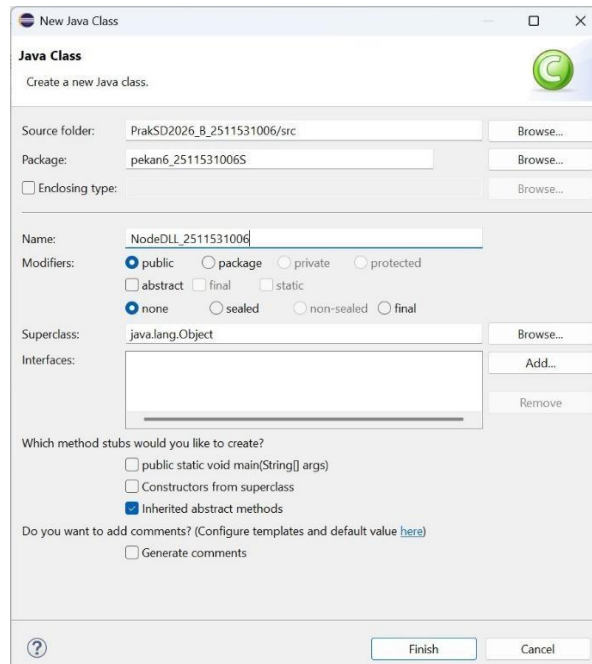
2.1 Langkah Langkah Praktikum

1. Program 1

- a. Pada tahap ini dilakukan pembuatan package baru pada project Java di Eclipse. Pengguna memilih menu New lalu memilih Package untuk membuat package baru. Package digunakan untuk mengelompokkan file Java agar lebih rapi dan terstruktur. Nama package yang dibuat adalah pekan6_2511531006S. Tahap ini dilakukan sebelum membuat class dan menuliskan kode program.



- b. Setelah package berhasil dibuat, langkah berikutnya adalah membuat class baru. Pengguna memilih menu New kemudian memilih Class pada Eclipse. Class digunakan sebagai tempat penulisan atribut dan method program Java. Pada tahap ini dibuat class bernama NodeDLL_2511531006. Class ini nantinya digunakan sebagai node pada Doubly Linked List.



c. Berikut merupakan kode awal class node

```
1 package pekan6_2511531006S;
2
3 public class NodeDLL_2511531006 {
```

Pada tahap awal di atas menunjukkan bahwa suatu class berada di package pekan6_2511531006S. Dan Class NodeDLL_2511531006 adalah digunakan dalam merepresentasikan node pada DLL. Modifier public membuat class dapat diakses oleh class lain. Class ini berfungsi menyimpan data dan pointer node. Node menjadi bagian utama dalam struktur linked list.

d. Selanjutnya program mendefinisikan atribut pada node.

```
4 // mendefinisikan kelas Node
5 int data_2511531006; // data
6 NodeDLL_2511531006 next_2511531006; // pointer ke next kode
7 NodeDLL_2511531006 prev_2511531006; // pointer ke previous node
8
```

Variabel `data_2511531006` digunakan untuk menyimpan data pada node. Variabel `next_2511531006` digunakan sebagai pointer ke node berikutnya. Variabel `prev_2511531006` digunakan sebagai pointer ke node sebelumnya. Kedua pointer tersebut menjadi ciri utama Doubly Linked List. Dengan atribut ini node dapat ditelusuri dua arah.

- e. Pada bagian berikutnya dibuat konstruktor class node.

```
9 // konstruktor
10 public NodeDLL_2511531006(int data_2511531006) {
```

Konstruktor digunakan untuk membuat objek node baru. Parameter `data_2511531006` digunakan untuk menerima data node. Konstruktor akan dijalankan otomatis saat objek dibuat. Modifier `public` membuat konstruktor dapat dipanggil dari class lain. Bagian ini penting dalam proses pembuatan node.

- f. Di dalam konstruktor dilakukan pengisian data node.

```
11 this.data_2511531006 = data_2511531006;
```

Kode ini digunakan untuk mengisi nilai data ke atribut node. `this` menunjukkan atribut milik class saat ini. Data yang diterima parameter dimasukkan ke variabel node. Dengan begitu setiap node memiliki data masing-masing. Proses ini terjadi saat objek node dibuat.

- g. Selanjutnya pointer `next` diinisialisasi menjadi `null`.

```
12 this.next_2511531006 = null;
```

Kode ini menandakan bahwa node belum memiliki node berikutnya. Nilai `null` berarti pointer belum terhubung ke node lain. Inisialisasi dilakukan saat node pertama kali

dibuat. Pointer next nantinya digunakan untuk traversal maju. Bagian ini penting dalam pembentukan linked list.

- h. Berikutnya pointer prev juga diinisialisasi menjadi null.

```
13         this.prev_2511531006 = null;
```

Kode ini menunjukkan bahwa node belum memiliki node sebelumnya. Nilai null berarti pointer belum terhubung ke node lain. Pointer prev digunakan untuk traversal mundur pada DLL. Inisialisasi awal membuat node aman sebelum dihubungkan. Bagian ini melengkapi struktur Doubly Linked List.

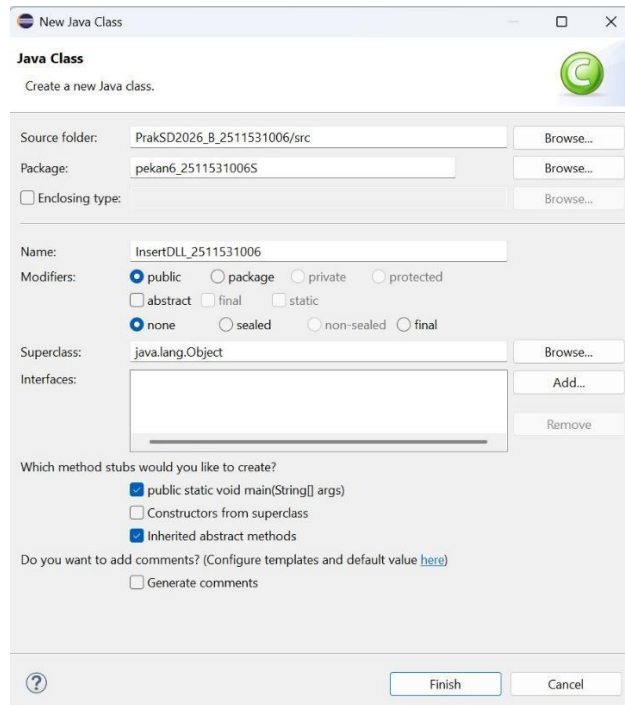
- i. Terakhir dilakukan penutupan konstruktor dan class.

```
14     }  
15  
16 }
```

Kurung kurawal pertama digunakan untuk menutup konstruktor. Kurung kurawal kedua digunakan untuk menutup class NodeDLL_2511531006. Penutupan class menandakan seluruh kode program telah selesai dibuat. Struktur class menjadi lengkap dan siap digunakan. Class node ini nantinya dipakai pada operasi Doubly Linked List lainnya

2. Program 2

- a. Setelah package berhasil dibuat, langkah berikutnya adalah membuat class baru. Pengguna memilih menu New kemudian memilih Class pada Eclipse. Class digunakan sebagai tempat penulisan atribut dan method program Java. Pada tahap ini dibuat class bernama InsertDLL_2511531006. Class ini nantinya digunakan sebagai node pada Doubly Linked List.



- b. Program diawali dengan pembuatan class utama menggunakan kode:

```
2  
3 public class InsertDLL_2511531006 {  
4
```

Class InsertDLL_2511531006 digunakan sebagai tempat seluruh method operasi Doubly Linked List. Di dalam class ini terdapat method untuk menambah node di awal, akhir, dan posisi tertentu. Class juga memiliki method untuk menampilkan isi linked list. Modifier public membuat class dapat diakses dari class lain. Program utama dijalankan melalui method main di dalam class ini.

- c. Pada bagian awal program dibuat method insertBegin untuk menambahkan node di awal linked list.

```

5 // menambahkan node di awal DLL
6⊖ static NodeDLL_2511531006 insertBegin(NodeDLL_2511531006 head_1006, int data_2511531006) {
7     // buat node baru
8     NodeDLL_2511531006 new_node_1006 = new NodeDLL_2511531006(data_2511531006);
9
10    // jadikan pointer next ke head
11    new_node_1006.next_2511531006 = head_1006;
12
13    // set prev head ke node baru
14⊖ if (head_1006 != null) {
15        head_1006.prev_2511531006 = new_node_1006;
16    }
17
18    return new_node_1006;
19 }

```

Method ini digunakan untuk menambahkan node baru di bagian depan linked list. Program membuat node baru lalu menghubungkannya ke head lama menggunakan pointer next. Jika linked list tidak kosong maka pointer prev dari head lama diarahkan ke node baru. Node baru kemudian dijadikan head terbaru pada linked list. Method ini mengembalikan head baru setelah proses insert selesai.

- d. Bagian berikutnya merupakan method insertEnd yang digunakan untuk menambahkan node di akhir linked list.

```

21 // menambahkan node di akhir DLL
22⊖ public static NodeDLL_2511531006 insertEnd(NodeDLL_2511531006 head_1006, int newData_2511531006) {
23     NodeDLL_2511531006 newNode_1006 = new NodeDLL_2511531006(newData_2511531006);
24
25⊖ if (head_1006 == null) {
26     head_1006 = newNode_1006;
27⊖ } else {
28     NodeDLL_2511531006 curr_1006 = head_1006;
29⊖ while (curr_1006.next_2511531006 != null) {
30         curr_1006 = curr_1006.next_2511531006;
31     }
32     curr_1006.next_2511531006 = newNode_1006;
33     newNode_1006.prev_2511531006 = curr_1006;
34 }
35 return head_1006;
36 }

```

Method ini digunakan untuk menambahkan node baru di bagian akhir linked list. Program terlebih dahulu membuat node baru menggunakan konstruktor node. Jika linked list kosong maka node baru langsung dijadikan head. Namun jika list sudah memiliki isi, program melakukan traversal hingga node terakhir ditemukan. Setelah itu node baru dihubungkan menggunakan pointer next dan prev.

- e. Selanjutnya program memiliki method insertAtPosition yang digunakan untuk menyisipkan node pada posisi tertentu.

```

38 // menambahkan node di posisi tertentu
39 public static NodeDLL_2511531006 insertAtPosition(
40     NodeDLL_2511531006 head_1006, int pos_1006, int new_data_1006) {
41
42     NodeDLL_2511531006 new_node_1006 = new NodeDLL_2511531006(new_data_1006);
43
44     if (pos_1006 == 1) {
45         new_node_1006.next_2511531006 = head_1006;
46         if (head_1006 != null) {
47             head_1006.prev_2511531006 = new_node_1006;
48         }
49         return new_node_1006;
50     }
51
52     NodeDLL_2511531006 curr_1006 = head_1006;
53     for (int i = 1; i < pos_1006 - 1 && curr_1006 != null; i++) {
54         curr_1006 = curr_1006.next_2511531006;
55     }
56
57     if (curr_1006 == null) {
58         System.out.println("Posisi tidak ada");
59         return head_1006;
60     }
61
62     new_node_1006.next_2511531006 = curr_1006.next_2511531006;
63     new_node_1006.prev_2511531006 = curr_1006;
64
65     if (curr_1006.next_2511531006 != null) {
66         curr_1006.next_2511531006.prev_2511531006 = new_node_1006;
67     }
68
69     curr_1006.next_2511531006 = new_node_1006;
70
71     return head_1006;
72 }

```

Method ini digunakan untuk menyisipkan node pada posisi tertentu di linked list. Program akan membuat node baru lalu mencari posisi sebelum tempat penyisipan. Jika posisi yang dimasukkan tidak ada maka program menampilkan pesan kesalahan. Node baru dihubungkan ke node sebelum dan sesudahnya menggunakan pointer next dan prev. Dengan begitu struktur Doubly Linked List tetap tersusun dengan benar.

- f. Pada bagian berikutnya dibuat method printList untuk menampilkan isi linked list.

```

74 public static void printList(NodeDLL_2511531006 head_1006) {
75     NodeDLL_2511531006 curr_1006 = head_1006;
76     while (curr_1006 != null) {
77         System.out.print(curr_1006.data_2511531006 + " <-> ");
78         curr_1006 = curr_1006.next_2511531006;
79     }
80     System.out.println("null");
81 }
82

```

Method printList digunakan untuk mencetak seluruh isi linked list ke layar. Program menggunakan variabel curr_1006 untuk melakukan traversal node. Perulangan while berjalan selama pointer belum bernilai null. Setiap data node

ditampilkan menggunakan simbol <-> sebagai penghubung. Null menandakan bahwa traversal sudah mencapai akhir list.

- g. Bagian terakhir program terdapat pada method main yang berfungsi sebagai tempat utama menjalankan program.

```
82  
83 public static void main(String[] args) {  
84
```

Method main merupakan method utama yang pertama kali dijalankan program. Semua proses pengujian Doubly Linked List dilakukan di dalam method ini. Program membuat linked list awal lalu melakukan beberapa operasi insert. Method ini juga digunakan untuk memanggil method lain seperti printList. Tanpa method main, program Java tidak dapat dijalankan.

- h. Di dalam method main, pertama-tama program membuat Doubly Linked List awal secara manual.

```
85 // membuat DLL: 2 <-> 3 <-> 5  
86 NodeDLL_2511531006 head_1006 = new NodeDLL_2511531006(2);  
87 head_1006.next_2511531006 = new NodeDLL_2511531006(3);  
88 head_1006.next_2511531006.prev_2511531006 = head_1006;  
89  
90 head_1006.next_2511531006.next_2511531006 = new NodeDLL_2511531006(5);  
91 head_1006.next_2511531006.next_2511531006.prev_2511531006 =  
92 head_1006.next_2511531006;
```

Kode tersebut digunakan untuk membuat linked list awal dengan isi 2 <-> 3 <-> 5. Node pertama disimpan dalam variabel head_1006. Node kedua dan ketiga kemudian dihubungkan menggunakan pointer next dan prev. Pointer next digunakan untuk hubungan maju sedangkan prev untuk hubungan mundur. Dengan demikian struktur Doubly Linked List berhasil dibuat secara lengkap.

- i. Setelah linked list awal selesai dibuat, program menampilkan isi linked list menggunakan kode:

```

93
94 // cetak DLL awal
95 System.out.print("DLL awal: ");
96 printList(head_1006);
97

```

Kode ini digunakan untuk mencetak kondisi awal linked list. Tulisan “DLL awal” digunakan sebagai keterangan output program. Method printList dipanggil untuk menampilkan seluruh node. Data linked list akan ditampilkan dari depan ke belakang. Output awal list adalah 2 <-> 3 <-> 5 <-> null.

- j. Selanjutnya program menambahkan node dengan data 1 di awal linked list menggunakan kode:

```

98 // tambah 1 di awal
99 head_1006 = insertBegin(head_1006, 1);
100 System.out.print("Simpul 1 ditambah di awal: ");
101 printList(head_1006);
102

```

Kode ini digunakan untuk menambahkan node baru di bagian depan linked list. Method insertBegin akan menjadikan node 1 sebagai head baru. Setelah proses insert selesai maka isi linked list berubah. Program kemudian menampilkan hasil linked list terbaru ke layar. Output list menjadi 1 <-> 2 <-> 3 <-> 5 <-> null.

- k. Berikutnya program menambahkan node dengan data 6 di akhir linked list menggunakan kode:

```

102 // tambah 6 di akhir
103 head_1006 = insertEnd(head_1006, 6);
104 System.out.print("Simpul 6 ditambah di akhir: ");
105 printList(head_1006);
106
107

```

Kode ini digunakan untuk menambahkan node baru di bagian akhir linked list. Method insertEnd melakukan traversal hingga node terakhir ditemukan. Node 6 kemudian dihubungkan ke node terakhir sebelumnya. Program mencetak kembali isi

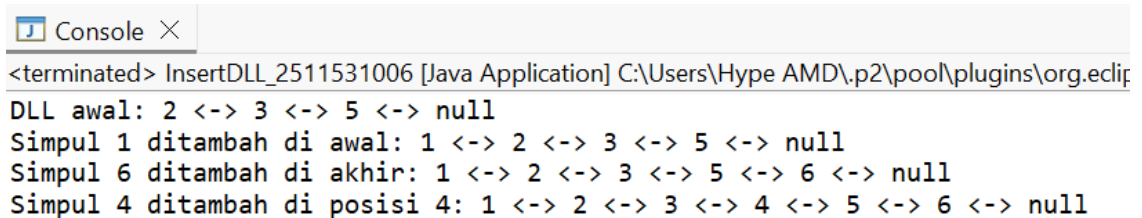
linked list setelah proses insert selesai. Output list berubah menjadi 1 <-> 2 <-> 3 <-> 5 <-> 6 <-> null.

- l. Terakhir program menyisipkan node dengan data 4 pada posisi keempat menggunakan kode:

```
108      // tambah 4 di posisi 4
109      head_1006 = insertAtPosition(head_1006, 4, 4);
110      System.out.print("Simpul 4 ditambah di posisi 4: ");
111      printList(head_1006);
112  }
113 }
```

Kode ini digunakan untuk menyisipkan node 4 pada posisi keempat. Method `insertAtPosition` mencari posisi sebelum tempat insert dilakukan. Node baru kemudian dihubungkan ke node sebelum dan sesudahnya. Setelah proses selesai program menampilkan hasil akhir linked list. Output akhirnya menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> null.

- m. output

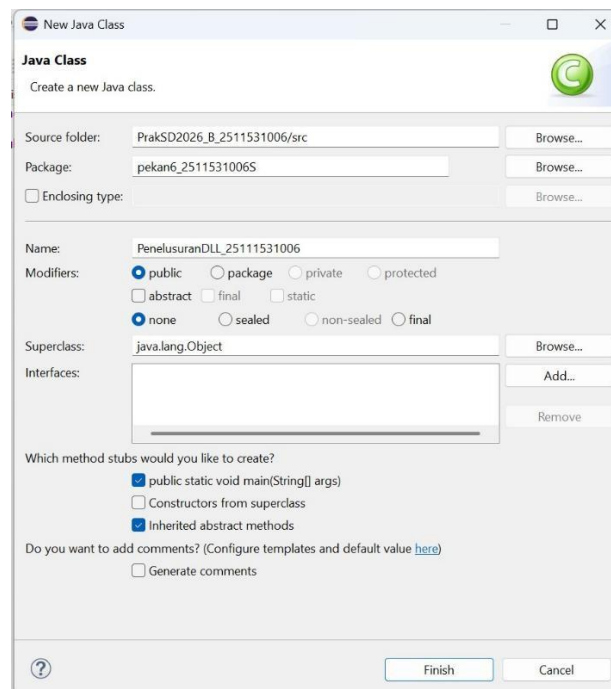


```
<terminated> InsertDLL_2511531006 [Java Application] C:\Users\Hype AMD\.p2\pool\plugins\org.eclipse
DLL awal: 2 <-> 3 <-> 5 <-> null
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <-> null
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <-> null
Simpul 4 ditambah di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> null
```

Berikut merupakan hasil output dari program Doubly Linked List yang telah dijalankan. Output pertama menampilkan linked list awal dengan isi 2 <-> 3 <-> 5 <-> null. Selanjutnya program berhasil menambahkan simpul 1 di awal sehingga data berubah menjadi 1 <-> 2 <-> 3 <-> 5 <-> null. Program juga berhasil menambahkan simpul 6 di akhir linked list dan menyisipkan simpul 4 pada posisi keempat. Hasil akhir menunjukkan linked list menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> null sehingga seluruh operasi insert berjalan dengan baik.

3. Program 3

- a. Pada tahap ini dilakukan pembuatan class baru pada project Java di Eclipse. Pengguna membuat class dengan nama `PenelusuranDLL_2511531006`. Class ini digunakan untuk melakukan traversal atau penelusuran data pada Doubly Linked List. Package yang digunakan adalah `pekan6_2511531006S` agar file program lebih rapi dan terstruktur. Method `main` juga dibuat otomatis untuk menjalankan program utama.



- b. Program diawali dengan penulisan package dan pembuatan class utama.

```
1 package pekan6_2511531006S;  
2  
3 public class PenelusuranDLL_2511531006 {
```

Kode ini menunjukkan bahwa class berada pada package `pekan6_2511531006S`. Class `PenelusuranDLL_2511531006` digunakan sebagai tempat operasi traversal DLL. Modifier `public` membuat class dapat diakses oleh class lain. Class ini berisi

method traversal maju dan traversal mundur. Selain itu terdapat method main sebagai pusat eksekusi program.

- c. Selanjutnya dibuat method forwardTraversal.

```
4 // fungsi penelusuran maju
5 static void forwardTraversal(NodeDLL_2511531006 head_1006) {
```

Method ini digunakan untuk melakukan penelusuran maju pada linked list. Traversal dimulai dari node pertama atau head linked list. Parameter head_1006 digunakan untuk menerima node awal. Method bertipe void karena hanya menampilkan output ke layar. Traversal dilakukan menggunakan pointer next.

- d. Di dalam method dibuat pointer traversal.

```
6 // memulai penelusuran dari head
7 NodeDLL_2511531006 curr_1006 = head_1006;
```

Variabel curr_1006 digunakan sebagai pointer sementara saat traversal. Pointer awal diarahkan ke node head linked list. Variabel ini akan berpindah dari satu node ke node berikutnya. Cara ini mempermudah proses penelusuran data. Pointer traversal sangat penting pada linked list.

- e. Program kemudian membuat perulangan traversal maju.

```
8 // lanjutan sampai akhir
9 while (curr_1006 != null) {
```

Perulangan digunakan untuk menelusuri seluruh node linked list. Loop akan berjalan selama pointer belum null. Artinya traversal dilakukan sampai node terakhir. Ketika pointer bernilai null, traversal akan berhenti. Perulangan ini umum digunakan pada struktur linked list.

- f. Di dalam loop program mencetak data node.

```
10 //print data
11 System.out.print(curr_1006.data_2511531006 + " <->");
```

Kode ini digunakan untuk menampilkan data setiap node ke layar. Data ditampilkan secara berurutan dari depan ke belakang. Simbol <-> menunjukkan hubungan antar node pada DLL. Output ditampilkan dalam satu baris traversal. Data node dicetak menggunakan System.out.print.

- g. Setelah data dicetak, pointer dipindahkan ke node berikutnya.

```
12 // pindah ke node berikutnya
13 curr_1006 = curr_1006.next_2511531006;
14 }
```

Kode ini memindahkan pointer ke node selanjutnya. Traversal maju menggunakan pointer next. Pointer akan terus bergerak sampai akhir linked list. Dengan begitu seluruh node dapat ditelusuri. Traversal selesai saat pointer bernilai null.

- h. Setelah traversal selesai program mencetak baris baru.

```
15 //print spasi
16 System.out.println();
17
```

Kode ini digunakan untuk membuat tampilan output lebih rapi. Cursor dipindahkan ke baris berikutnya setelah traversal selesai. Output traversal menjadi lebih mudah dibaca pengguna. Bagian ini menandakan akhir traversal maju. Program kemudian melanjutkan ke method berikutnya.

- i. Selanjutnya dibuat method backwardTraversal.

```
19 //fungsi penelusuran mundur
20 static void backwardTraversal(NodeDLL_2511531006 tail) {
```

Method ini digunakan untuk melakukan traversal mundur pada DLL.Traversal dimulai dari node terakhir atau tail linked list.Parameter tail digunakan sebagai node awal traversal mundur.Method bertipe void karena hanya menampilkan output.Traversal dilakukan menggunakan pointer prev.

- j. Di dalam method dibuat pointer traversal.

```
21 //mulai dari akhir
22 NodeDLL_2511531006 curr_1006 = tail;
```

Variabel curr_1006 digunakan sebagai pointer sementara saat traversal.Pointer awal diarahkan ke node head linked list.Variabel ini akan berpindah dari satu node ke node berikutnya.Cara ini mempermudah proses penelusuran data.Pointer traversal sangat penting pada linked list.

- k. Program kemudian membuat perulangan traversal maju.

```
23 //lanjut sampai head
24 while (curr_1006 !=null) {
```

Perulangan digunakan untuk menelusuri seluruh node linked list.Loop akan berjalan selama pointer belum bernilai null.Artinya traversal dilakukan sampai node terakhir.Ketika pointer bernilai null, traversal akan berhenti.Perulangan ini umum digunakan pada struktur linked list.

- l. Di dalam loop program mencetak data node.

```

25 //cetak data
26 System.out.print(curr_1006.data_2511531006 + " <->");

```

Kode ini digunakan untuk menampilkan data setiap node ke layar. Data ditampilkan secara berurutan dari depan ke belakang. Simbol <-> menunjukkan hubungan antar node pada DLL. Output ditampilkan dalam satu baris traversal. Data node dicetak menggunakan System.out.print.

- m. Setelah data dicetak, pointer dipindahkan ke node berikutnya.

```

27 //pindah ke node sebelumnya
28 curr_1006 = curr_1006.prev_2511531006;

```

Kode ini memindahkan pointer ke node selanjutnya. Traversal maju menggunakan pointer next. Pointer akan terus bergerak sampai akhir linked list. Dengan begitu seluruh node dapat ditelusuri. Traversal selesai saat pointer bernilai null.

- n. Setelah traversal selesai program mencetak baris baru.

```

30 //cetak spasi
31 System.out.println();
32 }

```

Kode ini digunakan untuk membuat tampilan output lebih rapi. Cursor dipindahkan ke baris berikutnya setelah traversal selesai. Output traversal menjadi lebih mudah dibaca pengguna. Bagian ini menandakan akhir traversal maju. Program kemudian melanjutkan ke method berikutnya.

- o. Selanjutnya dibuat method backwardTraversal.

```

19 //fungsi penelusuran mundur
20 static void backwardTraversal(NodeDLL_2511531006 tail) {

```

Method ini digunakan untuk melakukan traversal mundur pada DLL. Traversal dimulai dari node terakhir atau tail linked list. Parameter tail digunakan sebagai node awal traversal mundur. Method bertipe void karena hanya menampilkan output. Traversal dilakukan menggunakan pointer prev.

- p. Di dalam method dibuat pointer traversal mundur.

```
23 //lanjut sampai head
24 while (curr_1006 != null) {
```

Perulangan digunakan untuk menelusuri node dari belakang. Loop berjalan selama pointer belum bernilai null. Traversal dilakukan hingga mencapai node pertama. Ketika pointer null, loop akan berhenti otomatis. Perulangan ini membantu proses traversal data.

- q. Di dalam loop program mencetak data node.

```
25 //cetak data
26 System.out.print(curr_1006.data_2511531006 + " <->");
```

Kode ini digunakan untuk menampilkan data node secara mundur. Data dicetak mulai dari node terakhir ke node pertama. Simbol <-> menunjukkan hubungan antar node pada DLL. Output traversal tampil dalam satu baris. Data dicetak menggunakan perintah print.

- r. Setelah itu pointer dipindahkan ke node sebelumnya.

```
27 //pindah ke node sebelumnya
28 curr_1006 = curr_1006.prev_2511531006;
29 }
```

Kode ini memindahkan pointer ke node sebelumnya. Traversal mundur yaitu memanfaatkan pointer prev. Pointer akan bergerak ke belakang setiap perulangan. Hal ini ciri utama Doubly Linked List. Traversal selesai saat pointer bernilai null.

- s. Setelah traversal selesai program mencetak baris baru.

```
30 //cetak spasi
31 System.out.println();
```

Kode ini digunakan untuk merapikan tampilan output program. Cursor dipindahkan ke baris baru setelah traversal selesai. Output traversal mundur menjadi lebih mudah dibaca. Bagian ini menandakan akhir traversal mundur. Program kemudian masuk ke method utama.

- t. Bagian berikutnya merupakan method main.

```
33 public static void main(String[] args) {
```

Method main merupakan tempat utama eksekusi program Java. Semua proses pembuatan linked list dilakukan di bagian ini. Method ini juga digunakan untuk memanggil traversal maju dan mundur. Program pertama kali dijalankan dari method main. Tanpa method ini program tidak dapat dieksekusi.

- u. Di dalam method main program membuat node linked list.

```
34 //cetak DLL
35 NodeDLL_2511531006 head_1006 = new NodeDLL_2511531006(1);
36 NodeDLL_2511531006 second_2511531006 = new NodeDLL_2511531006(2);
37 NodeDLL_2511531006 third_2511531006 = new NodeDLL_2511531006(3);
38
```

Kode ini digunakan untuk membuat tiga node linked list. Node pertama berisi data 1, node kedua berisi 2, dan node ketiga berisi 3. Masing-masing node disimpan dalam

variabel berbeda. Node tersebut nantinya akan saling dihubungkan. Pembuatan node merupakan langkah awal pembentukan DLL.

- v. Setelah node dibuat, program menghubungkan seluruh node.

```
39         head_1006.next_2511531006 = second_2511531006;  
40         second_2511531006.prev_2511531006 = head_1006;  
41         second_2511531006.next_2511531006 = third_2511531006;  
42         third_2511531006.prev_2511531006 = second_2511531006;  
43
```

Kode ini digunakan untuk menghubungkan node satu dengan lainnya. Pointer next digunakan untuk hubungan maju antar node. Pointer prev digunakan untuk hubungan mundur antar node. Dengan begitu struktur Doubly Linked List berhasil terbentuk. Linked list yang dihasilkan adalah 1 <-> 2 <-> 3.

- w. Program kemudian menampilkan tulisan traversal maju.

```
43  
44         System.out.println("Penelusuran maju:");
```

Kode ini digunakan untuk memberi keterangan output traversal maju. Tulisan akan tampil sebelum data linked list dicetak. Hal ini membuat hasil program lebih mudah dipahami. Output menjadi lebih rapi dan terstruktur. Program kemudian memanggil method traversal maju.

- x. Berikutnya program memanggil method traversal maju.

```
45         forwardTraversal(head_1006);  
46
```

Method forwardTraversal dipanggil menggunakan parameter head. Program akan menampilkan data node pertama hingga terakhir. Traversal dilakukan menggunakan

pointer next. Output traversal maju akan muncul di console. Data yang tampil adalah 1 <-> 2 <-> 3.

- y. Program kemudian menampilkan tulisan traversal mundur. Dan program memanggil method traversal mundur.

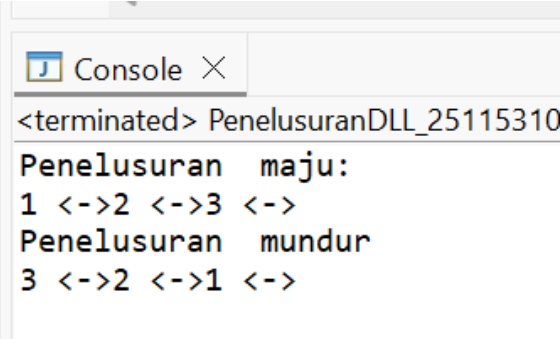
```
46  
47      System.out.println("Penelusuran  mundur");
```

Kode ini digunakan sebagai penjelas output traversal mundur. Tulisan akan muncul sebelum data traversal ditampilkan. Hal ini mempermudah pengguna membaca hasil program. Output menjadi lebih jelas dan rapi. Program selanjutnya memanggil traversal mundur.

```
48      backwardTraversal(third_2511531006);  
49
```

Method backwardTraversal dipanggil menggunakan node terakhir. Program akan menampilkan data dari belakang ke depan. Traversal dilakukan menggunakan pointer prev. Output traversal mundur menjadi 3 <-> 2 <-> 1. Hal ini membuktikan DLL dapat ditelusuri dua arah.

- z. Output

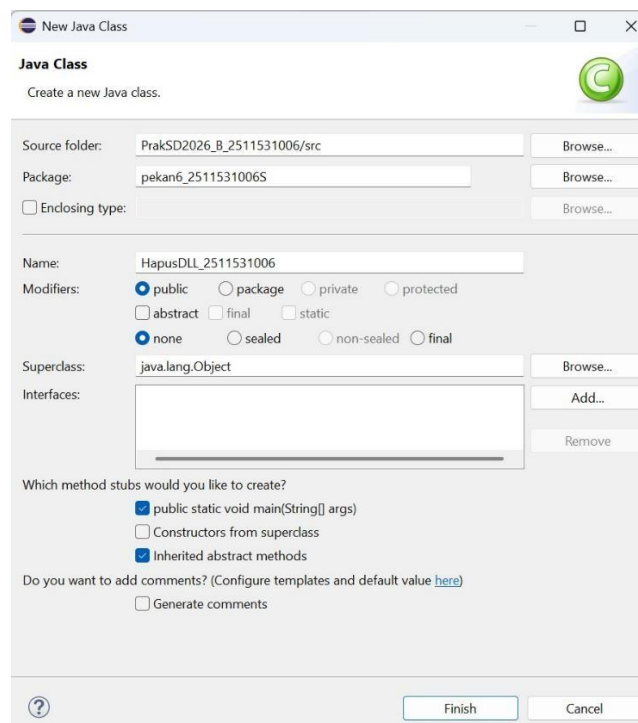


```
Console X  
<terminated> PenelusuranDLL_25115310  
Penelusuran  maju:  
1 <->2 <->3 <->  
Penelusuran  mundur  
3 <->2 <->1 <->
```

Output di atas menunjukkan hasil proses traversal atau penelusuran pada Doubly Linked List. Pada penelusuran maju, data ditampilkan mulai dari node pertama hingga node terakhir menggunakan pointer next, sehingga urutannya menjadi 1 <-> 2 <-> 3. Selanjutnya program melakukan penelusuran mundur menggunakan pointer prev dimulai dari node terakhir, sehingga data tampil menjadi 3 <-> 2 <-> 1. Simbol <-> digunakan untuk menunjukkan hubungan dua arah antar node pada Doubly Linked List. Hasil output membuktikan bahwa setiap node berhasil terhubung dengan baik ke node berikutnya maupun node sebelumnya.

4. Program 4

a. Pembuatan kelas baru



Pada tahap ini dilakukan pembuatan class baru pada Eclipse dengan nama HapusDLL_2511531006. Class ini digunakan untuk operasi penghapusan node pada Doubly Linked List. Package yang digunakan 24usic24 pekan6_2511531006S agar

file program lebih terstruktur. Method main dicentang agar program dapat langsung dijalankan. Tahap ini menjadi 25usic25n awal sebelum menuliskan kode program penghapusan node.

b. Pembuatan Class Utama

```
2  
3 public class HapusDLL_2511531006 {  
4
```

Kode ini untuk membuat class utama program. Class HapusDLL_2511531006 berfungsi sebagai tempat seluruh method penghapusan node. Di dalam class terdapat method hapus di awal, akhir, dan posisi tertentu. Modifier public membuat class dapat diakses dari class lain. Class ini juga memiliki method main sebagai pusat eksekusi program.

c. Method Penghapusan Head

```
4  
5 public static NodeDLL_2511531006 delHead(NodeDLL_2511531006 head_1006) {
```

Method delHead digunakan untuk menghapus node pertama pada Doubly Linked List. Parameter head_1006 digunakan menerima node awal linked list. Method bertipe NodeDLL_2511531006 karena mengembalikan head terbaru. Proses penghapusan dilakukan pada bagian depan linked list. Method ini penting untuk operasi dasar DLL.

d. Pengecekan Linked List Kosong

```
6  
7 if (head_1006 == null) {  
8     return null;  
9 }
```

Kode ini digunakan untuk mengecek apakah linked list kosong. Jika head bernilai null, berarti tidak ada node yang bisa dihapus. Program langsung mengembalikan nilai null. Pengecekan ini mencegah error saat program dijalankan. Validasi seperti ini penting dalam pengolahan linked list.

e. Memindahkan Head ke Node Berikutnya

```
9
10 NodeDLL_2511531006 temp_1006 = head_1006;
11 head_1006 = head_1006.next_2511531006;
12
```

Variabel temp_1006 digunakan untuk menyimpan head lama sementara. Kemudian head dipindahkan ke node berikutnya menggunakan pointer next. Artinya node pertama berhasil dilewati atau dihapus. Head baru sekarang menunjuk ke node setelahnya. Proses ini merupakan inti penghapusan node pertama.

f. Menghapus Hubungan Prev Head Baru

```
13
14 if (head_1006 != null) {
15     head_1006.prev_2511531006 = null;
16 }
```

Kode ini digunakan untuk menghapus hubungan prev pada head baru. Jika linked list masih memiliki isi, maka prev harus bernilai null. Hal ini karena node pertama tidak memiliki node sebelumnya. Proses ini menjaga struktur Doubly Linked List tetap benar. Pointer antar node tetap tersusun dengan baik.

g. Mengembalikan Head Baru

```
16
17     return head_1006;
18 }
```

Kode ini digunakan untuk mengembalikan head terbaru linked list. Head baru nantinya disimpan pada variabel utama program. Proses ini penting agar perubahan linked list tersimpan. Tanpa return, perubahan tidak akan terbaca di program utama. Bagian ini menandakan akhir method delHead.

h. Method Penghapusan Node Terakhir

```
19  
20 public static NodeDLL_2511531006 delLast(NodeDLL_2511531006 head_1006) {
```

Method ini digunakan untuk menghapus node terakhir linked list. Parameter head_1006 digunakan sebagai node awal traversal. Method akan mencari node terakhir sebelum dihapus. Penghapusan dilakukan menggunakan pointer prev dan next. Method ini termasuk operasi dasar pada DLL.

i. Pengecekan Linked List Kosong dan Satu Node

```
21 if (head_1006 == null) {  
22     return null;  
23 }  
24  
25 if (head_1006.next_2511531006 == null) {  
26     return null;  
27 }
```

Kode ini digunakan untuk validasi sebelum penghapusan dilakukan. Jika linked list kosong maka program langsung berhenti. Jika hanya ada satu node, maka setelah dihapus list menjadi kosong. Program mengembalikan null sebagai head baru.

j. Traversal Menuju Node Terakhir

```

28
29     NodeDLL_2511531006 curr_1006 = head_1006;
30
31     while (curr_1006.next_2511531006 != null) {
32         curr_1006 = curr_1006.next_2511531006;
33     }
34

```

Variabel curr_1006 digunakan sebagai pointer traversal. Program melakukan traversal dari head menuju node terakhir. Perulangan berjalan selama pointer next belum null. Ketika loop selesai, pointer akan berada di node terakhir. Traversal ini digunakan sebelum node terakhir dihapus.

k. Menghapus Node Terakhir

```

34
35     if (curr_1006.prev_2511531006 != null) {
36         curr_1006.prev_2511531006.next_2511531006 = null;
37     }
38

```

Kode ini digunakan untuk memutus hubungan node terakhir. Pointer next dari node sebelumnya diubah menjadi null. Artinya node terakhir tidak lagi terhubung ke linked list. Proses ini berhasil menghapus node paling belakang. Struktur linked list tetap tersusun dengan benar.

l. Method Penghapusan Posisi Tertentu

```

41
42     public static NodeDLL_2511531006 delPos(NodeDLL_2511531006 head_1006, int pos_1006) {
43

```

Method ini digunakan untuk menghapus node pada posisi suatu tertentu. Parameter pos_1006 digunakan untuk menentukan posisi node yang dihapus. Method menerima head sebagai titik awal traversal. Penghapusan dilakukan sesuai posisi input pengguna. Method ini membuat operasi DLL lebih fleksibel.

m. Traversal Mencari Posisi Node

```
47  
48 NodeDLL_2511531006 curr_1006 = head_1006;  
49  
50 for (int i = 1; curr_1006 != null && i < pos_1006; i++) {  
51     curr_1006 = curr_1006.next_2511531006;  
52 }  
53
```

Program menggunakan pointer curr_1006 untuk traversal node. Perulangan for digunakan untuk mencari posisi node tertentu. Pointer akan bergerak satu per satu hingga posisi ditemukan. Traversal berhenti saat node tujuan berhasil ditemukan. Cara ini umum digunakan pada linked list.

n. Pengecekan Posisi Tidak Ada

```
43  
44 if (head_1006 == null) {  
45     return head_1006;  
}
```

Kode ini digunakan untuk mengecek apakah posisi tersedia. Jika pointer bernilai null, berarti posisi tidak ditemukan. Program tidak melakukan penghapusan node. Head lama langsung dikembalikan. Hal ini mencegah error saat traversal.

o. Menghubungkan Node Sebelum dan Sesudah

```
58 if (curr_1006.prev_2511531006 != null) {  
59     curr_1006.prev_2511531006.next_2511531006 =  
60     curr_1006.next_2511531006;  
61 }
```

Kode ini digunakan untuk menghubungkan node sebelum node yang dihapus. Pointer next diarahkan ke node setelahnya. Dengan begitu node target berhasil

dilewati. Hubungan linked list tetap tersusun dengan benar. Pointer next menjadi bagian penting proses ini.

p. Mengatur Pointer Prev

```
62
63 if (curr_1006.next_2511531006 != null) {
64     curr_1006.next_2511531006.prev_2511531006 =
65         curr_1006.prev_2511531006;
66 }
```

Kode ini digunakan untuk mengatur pointer prev. Node setelah target diarahkan ke node sebelumnya. Hal ini akan menjaga hubungan dua arah antar node. Struktur Doubly Linked List tetap valid setelah penghapusan. Pointer prev menjadi ciri utama DLL.

q. Mengubah Head Jika Node Pertama Dihapus

```
67
68 if (head_1006 == curr_1006) {
69     head_1006 = curr_1006.next_2511531006;
70 }
71
```

Kode ini digunakan jika node yang dihapus adalah head. Head dipindahkan ke node berikutnya pada linked list. Dengan begitu linked list tetap memiliki head yang benar. Perubahan head penting agar list tetap dapat diakses. Bagian ini melengkapi proses penghapusan posisi.

r. Method Menampilkan Linked List

```
74
75 public static void printList_1006(NodeDLL_2511531006 head_1006) {
76
```

Method `printList_1006` digunakan untuk menampilkan isi linked list. Traversal dimulai dari head hingga node terakhir. Method bertipe void karena hanya mencetak output. Data setiap node akan ditampilkan ke 31usid. Method ini membantu melihat hasil perubahan list.

s. Traversal dan Cetak Data

```
78
79 while (curr_1006 != null) {
80     System.out.print(curr_1006.data_2511531006 + " ");
81     curr_1006 = curr_1006.next_2511531006;
82 }
83
```

Perulangan digunakan untuk menampilkan seluruh isi linked list. Data node dicetak satu per satu dari depan ke belakang. Pointer bergerak menggunakan next. Traversal berhenti 31usid31 pointer bernilai null. Output linked list akan tampil pada console.

t. Method Main Program

```
86
87 public static void main(String[] args) {
88
```

Method main merupakan pusat utama eksekusi program Java. Semua proses dalam pembuatan linked list dilakukan di bagian ini. Method juga digunakan untuk memanggil operasi penghapusan node. Program pertama kali berjalan dari method main. Bagian ini menjadi inti jalannya program.

u. Membuat Linked List Awal

```
88
89 NodeDLL_2511531006 head_1006 =
90     new NodeDLL_2511531006(1);
91
```

Kode ini digunakan untuk membuat node pertama linked list. Node pertama memiliki data bernilai 1. Variabel head_1006 menjadi awal linked list. Node dibuat dengan menggunakan konstruktor class node. Bagian ini menjadi awal pembentukan DLL.

v. Menghubungkan Node Hingga Data 5

```
91  
92  
93  
94  
head_1006.next_2511531006 =  
new NodeDLL_2511531006(2);
```

Kode berikutnya digunakan untuk membuat node kedua hingga kelima. Setiap node dihubungkan menggunakan pointer next dan prev. Hubungan dua arah akan membuat traversal bisa maju dan mundur. Linked list yang terbentuk 32usic32 1 2 3 4 5. Struktur DLL berhasil dibuat dengan lengkap.

w. Menampilkan DLL Awal

```
115  
116  
117  
118  
System.out.print("DLL Awal: ");  
printList_1006(head_1006);
```

Kode ini digunakan untuk menampilkan isi linked list awal. Output ditampilkan sebelum proses penghapusan dilakukan. Method printList_1006 dipanggil untuk mencetak data node. Pengguna dapat melihat kondisi awal linked list. Data awal yang tampil 32usic32 1 2 3 4 5.

x. Menghapus Node Head

```
118  
119  
120  
121  
System.out.print("Setelah head dihapus: ");  
head_1006 = delHead(head_1006);  
printList_1006(head_1006);
```


Method delHead dipanggil untuk menghapus node pertama. Node dengan data 1 akan dihapus dari linked list. Head baru berpindah ke node berikutnya. Linked list berubah menjadi 2 3 4 5. Hasil perubahan kemudian ditampilkan ke 33usic.

y. Menghapus Node Terakhir dan menghapus node posisi tertentu

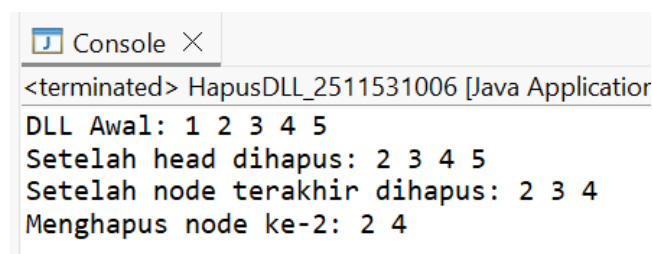
```
122  
123  
124  
125  
126  
System.out.print("Setelah node terakhir dihapus: ");  
head_1006 = delLast(head_1006);  
printList_1006(head_1006);
```

Method delLast digunakan untuk menghapus node terakhir. Node paling belakang pada linked list akan dihapus. Setelah proses selesai linked list menjadi lebih pendek. Data terakhir yang sebelumnya 5 berhasil dihapus. Hasil list kemudian ditampilkan 33usic33n.

```
120  
127  
128  
129  
130  
System.out.print("Menghapus node ke-2: ");  
head_1006 = delPos(head_1006, 2);  
printList_1006(head_1006);  
}
```

Method delPos digunakan untuk menghapus node pada posisi tertentu. Pada kode ini node posisi kedua akan dihapus dari linked list. Traversal dilakukan terlebih dahulu untuk mencari posisi node. Setelah penghapusan selesai linked list berubah 33usic33n. Program kemudian menampilkan hasil akhir linked list ke console.

z. Output



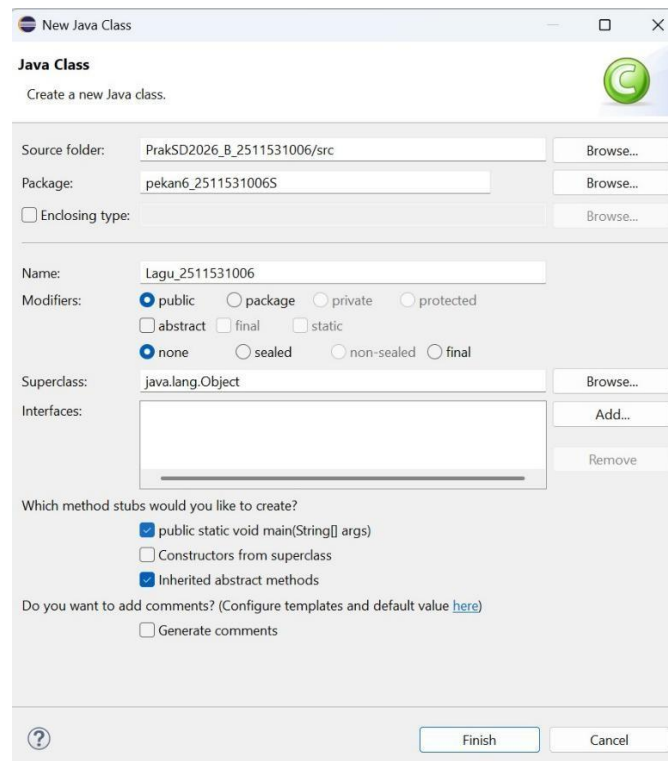
```
Console X  
<terminated> HapusDLL_2511531006 [Java Application]  
DLL Awal: 1 2 3 4 5  
Setelah head dihapus: 2 3 4 5  
Setelah node terakhir dihapus: 2 3 4  
Menghapus node ke-2: 2 4
```

Output di atas menunjukkan hasil dari setiap proses penghapusan node pada Doubly Linked List. Pada tampilan awal, linked list berisi data 1 2 3 4 5 yang dibuat secara berurutan. Setelah method delHead dijalankan, node pertama yaitu 1 berhasil dihapus sehingga data berubah menjadi 2 3 4 5. Selanjutnya method delLast menghapus node terakhir yaitu 5, sehingga linked list menjadi 2 3 4. Pada proses terakhir, method delPos digunakan untuk menghapus node pada posisi ke-2 yaitu data 3, sehingga hasil akhir linked list menjadi 2 4.

2.2 Langkah Langkah Tugas Praktikum

1. Program 5

a. Membuat Class Baru di Eclipse



Pada tahap ini dilakukan proses pembuatan class baru di Eclipse dengan memilih menu New → Class. Langkah tersebut digunakan untuk membuat file Java baru yang

nantinya akan berisi program Doubly Linked List lagu. Pada bagian Name diisi nama class yaitu Lagu_2511531006 sesuai nama program yang akan dibuat. Package yang digunakan 35usic35 pekan6_2511531006S sehingga class tersimpan dalam package yang sama dengan file lainnya. Selain itu opsi public static void main(String[] args) dicentang agar Eclipse otomatis membuat method utama untuk menjalankan program.

b. Membuat Class Lagu_2511531006

```
2  
3 public class Lagu_2511531006 {  
4
```

Kode di atas digunakan untuk membuat class 35usic35n Lagu_2511531006. Class ini berfungsi sebagai representasi node pada Doubly Linked List yang menyimpan data lagu. Di dalam class terdapat atribut untuk menyimpan judul lagu, trus nama penyanyi, serta pointer next dan prev.Pointer tersebut digunakan yaitu untuk untuk menghubungkan node satu dengan node lainnya secara dua arah. Penggunaan class ini membuat data lagu dapat dikelola dengan lebih terstruktur dalam program.

c. Mendeklarasikan Variabel Data dan Pointer

```
5 public String judul_1006;  
6 public String penyanyi_1006;  
7 public Lagu_2511531006 next_1006;  
8 public Lagu_2511531006 prev_1006;  
9
```

Bagian ini digunakan untuk mendeklarasikan atribut yang terdapat pada node lagu. Variabel judul_1006 digunakan untuk menyimpan judul lagu, sedangkan penyanyi_1006 digunakan untuk menyimpan nama penyanyi.Selain itu terdapat pointer next_1006 yang menunjuk ke node berikutnya dan prev_1006 yang menunjuk ke node sebelumnya. Kedua pointer tersebut merupakan ciri utama dari struktur

Doubly Linked List karena setiap node saling terhubung dua arah. Penggunaan modifier public membuat seluruh atribut dapat diakses dari class lain.

d. Membuat Constructor Lagu

```
10      // Constructor
11      public Lagu_2511531006(String judul_1006, String penyanyi_1006) {
12          this.judul_1006 = judul_1006;
13          this.penyanyi_1006 = penyanyi_1006;
14          this.next_1006 = null;
15          this.prev_1006 = null;
16      }
```

Constructor digunakan untuk menginisialisasi objek baru. Node lagu dibuat. Parameter judul_1006 dan penyanyi_1006 akan langsung disimpan ke dalam atribut class menggunakan keyword this. Selain itu pointer next_1006 dan prev_1006 diatur bernilai null karena node baru belum terhubung dengan node lain. Constructor membantu proses pembuatan node menjadi lebih mudah dan rapi. Dengan adanya constructor, data lagu dapat langsung dimasukkan saat objek dibuat.

e. Membuat Method Getter

```
18      // Getter
19      public String getJudul_1006() { return judul_1006; }
20      public String getPenyanyi_1006() { return penyanyi_1006; }
21      public Lagu_2511531006 getNext_1006() { return next_1006; }
22      public Lagu_2511531006 getPrev_1006() { return prev_1006; }
23
```

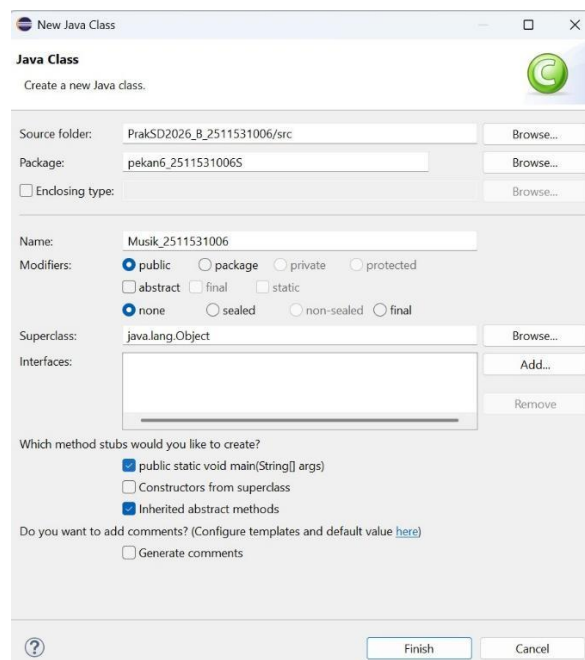
Method getter digunakan untuk mengambil atau membaca nilai dari atribut yang terdapat pada class. Method getJudul_1006 digunakan untuk mengambil judul lagu, sedangkan getPenyanyi_1006 digunakan untuk mengambil nama penyanyi. Selain itu terdapat getter untuk pointer next_1006 dan prev_1006 agar hubungan antar node dapat diakses dari luar class. Penggunaan getter membuat data lebih aman karena akses dilakukan melalui method. Ini juga merupakan bagian dari konsep enkapsulasi pada pemrograman berorientasi objek.

f. Membuat Method Setter

```
24 // Setter
25 public void setJudul_1006(String judul_1006) { this.judul_1006 = judul_1006; }
26 public void setPenyanyi_1006(String penyanyi_1006) { this.penyanyi_1006 = penyanyi_1006; }
27 public void setNext_1006(Lagu_2511531006 next_1006) { this.next_1006 = next_1006; }
28 public void setPrev_1006(Lagu_2511531006 prev_1006) { this.prev_1006 = prev_1006; }
29
30 }
```

2. Program 6

a. Membuat Class Baru di Eclipse



Pada tahap ini proses pembuatan class baru di Eclipse dengan nama Musik_2511531006. Class ini dibuat di dalam package pekan6_2511531006S yang digunakan untuk menyimpan program playlist music berbasis Doubly Linked List. Pada bagian “Which method stubs would you like to create?”, opsi public static void main(String[] args) dicentang agar Eclipse otomatis membuat method main sebagai tempat utama menjalankan program. Modifier public dipilih supaya class diakses dari

class lain di dalam project. Setelah seluruh pengaturan selesai, tombol Finish ditekan untuk membuat file class baru. Program diawali dengan deklarasi package dan import library menggunakan kode berikut.

```
1 package pekan6_2511531006S;  
2  
3 import java.util.Scanner;  
4
```

Kode package pekan6_2511531006S digunakan untuk menempatkan class ke dalam package tertentu agar file program lebih rapi dan mudah dikelompokkan. Selanjutnya program mengimpor library Scanner dari package java.util yang berfungsi untuk menerima input dari keyboard. Library ini nantinya dipakai pada menu program agar pengguna dapat memasukkan pilihan, judul lagu, dan nama penyanyi. Penggunaan package dan import membantu program menjadi lebih terstruktur dan mudah dikembangkan. Dengan adanya Scanner, program dapat berjalan secara interaktif melalui console.

- b. Berikutnya dibuat class utama Musik_2511531006.

```
5 public class Musik_2511531006 {  
6
```

Class Musik_2511531006 berfungsi sebagai class utama untuk mengelola playlist music menggunakan struktur data Doubly Linked List. Di dalam class ini terdapat berbagai method seperti menambah lagu, menghapus lagu, menampilkan playlist maju dan mundur, serta mencari lagu. Modifier public digunakan agar class dapat diakses dari file lain pada project. Seluruh proses pengolahan playlist dilakukan di dalam class ini sehingga program lebih terorganisir. Class ini juga memiliki method main sebagai pusat eksekusi program.

- c. Pada bagian berikutnya program mendeklarasikan lagu head dan tail.

```
6  
7     private Lagu_2511531006 head_1006;  
8     private Lagu_2511531006 tail_1006;  
9
```

Variabel head_1006 digunakan untuk menyimpan node pertama pada Doubly Linked List, sedangkan tail_1006 digunakan untuk menyimpan node terakhir. Kedua music ini bertipe Lagu_2511531006 karena setiap node berisi data lagu. Modifier private digunakan agar data hanya dapat diakses di dalam class sehingga keamanan data lebih terjaga. Dengan adanya head dan tail, proses penelusuran maju maupun mundur menjadi lebih mudah dilakukan. Variabel ini merupakan bagian penting dalam struktur Doubly Linked List.

- d. Selanjutnya dibuat constructor class.

```
10 public Musik_2511531006() {  
11     head_1006 = null;  
12     tail_1006 = null;  
13 }  
14
```

Constructor Musik_2511531006() digunakan untuk menginisialisasi linked list saat object pertama kali dibuat. Pada awal program, nilai head_1006 dan tail_1006 diatur menjadi null yang menandakan playlist masih kosong dan belum memiliki node. Constructor berjalan otomatis music object dibuat menggunakan keyword new. Dengan constructor ini, program memiliki kondisi awal yang jelas sebelum data dimasukkan. Hal ini penting agar tidak terjadi error musik proses penambahan data dilakukan.

- e. Bagian berikut merupakan method tambahLagu_1006().

```
15 // 1. tambahLagu_1006()  
16 public void tambahLagu_1006(String judul, String penyanyi) {
```

Method tambahLagu_1006() digunakan untuk menambahkan lagu 40usid40n dalam playlist. Method menerima dua parameter yaitu judul dan penyanyi yang nantinya disimpan ke dalam node baru bertipe Lagu_2511531006. Jika playlist masih kosong, maka node baru langsung dijadikan head dan tail. Namun jika playlist sudah memiliki isi, node baru akan ditambahkan di bagian akhir linked list.

- f. Pada bagian isi method dilakukan pembuatan node baru.

```
17      Lagu_2511531006 baru = new Lagu_2511531006(judul, penyanyi);
```

Kode ini digunakan untuk membuat object node baru. Node tersebut akan menyimpan data judul lagu dan nama penyanyi sesuai input pengguna. Constructor pada class Lagu_2511531006 dipanggil untuk mengisi data sekaligus mengatur pointer next dan prev menjadi null. Setiap lagu yang dimasukkan pengguna akan dibuat menjadi node baru seperti ini. Dengan cara tersebut, data playlist dapat disimpan secara dinamis menggunakan linked list.

- g. Berikutnya dilakukan pengecekan apakah playlist kosong atau tidak.

```
18      if (head_1006 == null) {  
19          head_1006 = tail_1006 = baru;
```

Kondisi ini digunakan untuk mengecek apakah linked list masih kosong. Jika head_1006 bernilai null, maka node baru langsung dijadikan node pertama sekaligus node terakhir pada playlist. Hal ini karena pada saat awal hanya terdapat satu node saja. Pointer head dan tail akan menunjuk ke node yang sama.

- h. Jika playlist sudah memiliki isi maka dijalankan bagian berikut.


```

20-         } else {
21             tail_1006.setNext_1006(baru);
22             baru.setPrev_1006(tail_1006);
23             tail_1006 = baru;
24         }

```

Kode ini digunakan untuk menambahkan lagu baru di bagian akhir linked list. Pointer next dari node terakhir diarahkan ke node baru menggunakan method setter. Kemudian pointer prev pada node baru diarahkan ke node sebelumnya agar hubungan dua arah tetap terbentuk. Setelah itu tail_1006 dipindahkan ke node baru karena sekarang node tersebut menjadi node terakhir. Dengan proses ini penambahan lagu pada playlist berhasil dilakukan dengan benar.

- i. Selanjutnya program menampilkan pesan berhasil.

```

25         System.out.println("Lagu berhasil ditambahkan!");
26     }
27

```

Kode ini digunakan untuk memberikan informasi kepada pengguna bahwa lagu berhasil dimasukkan ke playlist. Pesan ditampilkan di console setelah proses penambahan node selesai dilakukan. Dengan adanya output ini pengguna dapat mengetahui bahwa operasi berjalan tanpa error. Tampilan pesan juga membuat program menjadi lebih interaktif. Hal ini membantu pengguna saat menggunakan menu program.

- j. Bagian berikut merupakan method hapusLaguAwal_1006().

```

28         // 2. hapusLaguAwal_1006()
29-     public void hapusLaguAwal_1006() {

```

Method ini digunakan untuk menghapus node pertama pada playlist music. Pada Penghapusan dilakukan pada bagian head linked list sehingga lagu pertama akan

hilang dari playlist. Method ini tidak memerlukan parameter karena langsung bekerja pada node pertama. Dalam Doubly Linked List, proses penghapusan harus memperhatikan hubungan pointer antar node. Oleh karena itu program melakukan beberapa pengecekan sebelum penghapusan dilakukan.

- k. Pada bagian awal method dilakukan pengecekan playlist kosong.

```
30-      if (head_1006 == null) {  
31          System.out.println("Playlist kosong!");  
32          return;  
33      }
```

Kode ini digunakan untuk memastikan apakah playlist memiliki data atau tidak. Jika head_1006 bernilai null, maka artinya playlist kosong sehingga tidak ada node yang bisa dihapus. Program kemudian menampilkan pesan "Playlist kosong!" dan method dihentikan menggunakan return. Pengecekan ini penting agar program tidak mengalami error saat mencoba menghapus node yang tidak ada. Dengan demikian program menjadi lebih aman saat dijalankan.

- l. Selanjutnya program mengecek apakah hanya ada satu lagu.

```
34-      if (head_1006 == tail_1006) {  
35          head_1006 = tail_1006 = null;
```

Kondisi ini digunakan music playlist hanya memiliki satu node saja. Jika head dan tail menunjuk node yang sama, maka setelah dihapus playlist akan menjadi kosong. Oleh karena itu nilai head dan tail diubah menjadi null.

- m. Jika playlist memiliki lebih dari satu node maka dijalankan bagian berikut.

```
36-      } else {  
37          head_1006 = head_1006.getNext_1006();  
38          head_1006.setPrev_1006(null);  
39      }
```

Kode ini digunakan untuk memindahkan head ke node berikutnya setelah node pertama dihapus. Setelah head berpindah, pointer prev pada head baru diatur menjadi null karena sekarang node tersebut berada di posisi paling depan.

- n. Bagian berikut merupakan method tampilMaju_1006().

```
43 // 3. tampilMaju_1006()  
44 public void tampilMaju_1006() {
```

Method tampilMaju_1006() digunakan untuk menampilkan seluruh isi playlist dari depan ke belakang. Penelusuran dilakukan mulai dari head hingga node terakhir menggunakan pointer next. Method ini membantu pengguna melihat urutan lagu sesuai data yang dimasukkan. Jika playlist kosong maka program akan menampilkan pesan khusus. Penelusuran maju merupakan salah satu ciri utama struktur Doubly Linked List.

- o. Selanjutnya program mencetak isi playlist.

```
52 while (tmp != null) {  
53     System.out.printf("%d. %s - %s%n", i++, tmp.getJudul_1006(), tmp.getPenyanyi_1006());  
54     tmp = tmp.getNext_1006();  
55 }  
56  
57
```

Perulangan while digunakan untuk menelusuri seluruh node pada linked list. Selama pointer tmp belum bernilai null, program akan menampilkan nomor urut, judul lagu, dan nama penyanyi. Setelah data ditampilkan, pointer dipindahkan ke node berikutnya menggunakan getNext_1006(). Proses ini terus dilakukan hingga mencapai node terakhir.

- p. Berikutnya terdapat method tampilMundur_1006().

```

58 // 4. tampilMundur_1006()
59 public void tampilMundur_1006() {

```

Method ini digunakan untuk menampilkan playlist dari belakang ke depan. Penelusuran dimulai dari tail dan bergerak menggunakan pointer prev. Method ini menunjukkan keunggulan Doubly Linked List karena data dapat ditelusuri dua arah. Jika playlist kosong maka program akan menampilkan pesan bahwa data tidak tersedia. Dengan method ini pengguna dapat melihat urutan lagu secara terbalik.

- q. Pada bagian penelusuran mundur digunakan kode berikut.

```

67 while (tmp != null) {
68     System.out.printf("%d. %s - %s\n", i++, tmp.getJudul_1006(), tmp.getPenyanyi_1006());
69     tmp = tmp.getPrev_1006();
70 }
71
72

```

Perulangan ini digunakan untuk menampilkan isi playlist dari belakang ke depan. Pointer tmp dimulai dari tail lalu bergerak menuju node sebelumnya menggunakan getPrev_1006(). Setiap node yang dilewati akan ditampilkan ke console lengkap dengan nomor urut, judul lagu, dan nama penyanyi.

- r. Bagian berikut merupakan method cariLagu_1006().

```

73 // 5. cariLagu_1006(judul)
74 public void cariLagu_1006(String judul) {

```

Method cariLagu_1006() digunakan untuk mencari lagu berdasarkan judul yang dimasukkan pengguna. Method menerima parameter berupa teks judul lagu yang ingin dicari. Program akan melakukan traversal dari head hingga akhir linked list. Jika lagu ditemukan maka data lagu akan ditampilkan di console. Method ini mempermudah pengguna mencari lagu tertentu di dalam playlist.

- s. Pada proses pencarian digunakan kondisi berikut.

```
82 if (tmp.getJudul_1006().toLowerCase().contains(judul.toLowerCase())) {
```

Kode ini digunakan untuk membandingkan input pengguna dengan judul lagu yang ada pada playlist. Method `toLowerCase()` dipakai agar pencarian tidak membedakan huruf besar dan kecil. Sedangkan `contains()` digunakan agar pengguna dapat mencari kata dari judul lagu. Jika judul sesuai maka program akan menampilkan lagu yang ditemukan.

- t. Bagian terakhir program terdapat pada method main.

```
91  
92 public static void main(String[] args) {
```

Method main merupakan tempat utama menjalankan seluruh program playlist music. Di dalam method ini dibuat object Scanner untuk menerima input pengguna dan object Musik_2511531006 untuk menjalankan operasi playlist. Program menggunakan menu berbasis angka agar pengguna dapat memilih fitur yang tersedia. Seluruh proses dijalankan menggunakan perulangan `do while`. Dengan method ini program dapat berjalan secara interaktif melalui console.

- u. Pada bagian menu digunakan kode berikut.

```
System.out.println("\n=== Playlist Musik NIM: 2511531006 ===");  
System.out.println("1. Tambah Lagu");  
System.out.println("2. Hapus Lagu Pertama");  
System.out.println("3. Lihat Playlist (Maju)");  
System.out.println("4. Lihat Playlist (Mundur)");  
System.out.println("5. Cari Lagu");  
System.out.println("6. Keluar");  
System.out.println();  
System.out.print("Pilihan: ");
```

Kode tersebut digunakan untuk menampilkan menu utama program playlist music. Pengguna dapat memilih fitur sesuai angka yang tersedia pada console. Menu dibuat

sederhana agar mudah digunakan saat program dijalankan. Setiap pilihan nantinya akan diproses menggunakan switch case.

- v. Berikutnya program menggunakan switch case.

```
109  
110 switch (pilihan) {
```

Struktur switch case digunakan untuk menjalankan fitur sesuai pilihan pengguna. Setiap angka pada menu memiliki proses yang berbeda seperti menambah lagu, menghapus lagu, menampilkan playlist, dan mencari lagu. Jika pengguna memasukkan angka yang tidak tersedia maka program akan menampilkan pesan “Pilihan tidak valid!”. Penggunaan switch case membuat kode lebih rapi dibanding banyak if else.

- w. Pada bagian akhir program terdapat perulangan berikut.

```
128 } while (pilihan != 6);
```

Perulangan do while digunakan agar menu terus muncul sampai pengguna memilih angka 6 untuk keluar. Selama pilihan bukan 6, program akan terus berjalan dan menerima input baru dari pengguna. Dengan metode ini pengguna dapat menjalankan berbagai fitur tanpa harus membuka ulang program. Setelah memilih keluar, program menampilkan pesan “Terima kasih!”. Program kemudian berhenti dan Scanner ditutup menggunakan `sc.close()`.

- x. Output



```
Console X
Musik_2511531006 [Java Application] C:\Users\Hype AI

=== Playlist Musik NIM: 2511531006 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1
Judul: Raim Laode - Kenangan Manis
Penyanyi: Raim Laode
Lagu berhasil ditambahkan!
```

Pada tampilan console di atas, program pertama kali menampilkan menu utama Playlist Musik berbasis Doubly Linked List dengan identitas NIM 2511531006. Menu tersebut berisi beberapa pilihan, seperti menambah lagu, menghapus lagu pertama, melihat playlist secara maju maupun mundur, mencari lagu, dan keluar dari program. Pada proses ini, pengguna memilih menu nomor 1 yaitu Tambah Lagu dengan memasukkan judul lagu “Raim Laode – Kenangan Manis”. Setelah itu, program meminta input nama penyanyi dan pengguna mengisi “Raim Laode”. Data lagu yang telah diinput kemudian disimpan ke dalam playlist menggunakan proses penambahan node pada bagian akhir Doubly Linked List. Setelah proses berhasil dilakukan, program menampilkan pesan “Lagu berhasil ditambahkan!” sebagai tanda bahwa data lagu sudah berhasil masuk ke dalam playlist music.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Doubly Linked List merupakan struktur data yang memiliki dua arah penelusuran, yaitu maju dan mundur. Struktur data ini menggunakan pointer *next* dan *prev* untuk menghubungkan setiap node. Dalam praktikum ini telah dibuat beberapa program seperti pembuatan node, insert node, delete node, traversal maju dan mundur, serta program playlist musik menggunakan bahasa Java. Dari hasil praktikum dapat dipahami bahwa DLL mempermudah proses manipulasi data karena node dapat diakses dari dua arah. Selain itu, implementasi playlist musik menunjukkan bahwa konsep Doubly Linked List dapat digunakan dalam kehidupan sehari-hari, terutama pada aplikasi yang membutuhkan navigasi data secara fleksibel.

3.2 Saran

Dalam pembuatan program Doubly Linked List diperlukan ketelitian dalam menghubungkan pointer *next* dan *prev* agar tidak terjadi error pada program. Mahasiswa disarankan untuk lebih banyak berlatih membuat program linked list agar lebih memahami alur kerja struktur data dinamis.

DAFTAR PUSTAKA

GeeksforGeeks - Doubly Linked List Introduction

Programiz - Doubly Linked List Data Structure